
Overcoming Sparsity Artifacts in Crosscoders to Interpret Chat-Tuning

Julian Minder^{*⊙ⓐ} Clément Dumas^{*†‡}
Caden Juang[ⓓ] Bilal Chughtai Neel Nanda

[⊙]EPFL [ⓐ]ETHZ [†]Ecole Normale Supérieure Paris-Saclay [‡]Université Paris-Saclay

[ⓓ]Northeastern University

julian.minder@epfl.ch, clement.dumas@ens-paris-saclay.fr

Abstract

Model diffing is the study of how fine-tuning changes a model’s representations and internal algorithms. Many behaviors of interest are introduced during fine-tuning, and model diffing offers a promising lens to interpret such behaviors. Crosscoders are a recent model diffing method that learns a shared dictionary of interpretable concepts represented as latent directions in both the base and fine-tuned models, allowing us to track how concepts shift or emerge during fine-tuning. Notably, prior work has observed concepts with no direction in the base model, and it was hypothesized that these model-specific latents were concepts introduced during fine-tuning. However, we identify two issues which stem from the crosscoders L1 training loss that can misattribute concepts as unique to the fine-tuned model, when they really exist in both models. We develop Latent Scaling to flag these issues by more accurately measuring each latent’s presence across models. In experiments comparing Gemma 2 2B base and chat models, we observe that the standard crosscoder suffers heavily from these issues. Building on these insights, we train a crosscoder with BatchTopK loss and show that it substantially mitigates these issues, finding more genuinely chat-specific and highly interpretable concepts. We recommend practitioners adopt similar techniques. Using the BatchTopK crosscoder, we successfully identify a set of chat-specific latents that are both interpretable and causally effective, representing concepts such as *false information* and *personal question*, along with multiple refusal-related latents that show nuanced preferences for different refusal triggers. Overall, our work advances best practices for the crosscoder-based methodology for model diffing and demonstrates that it can provide concrete insights into how chat-tuning modifies model behavior.¹

1 Introduction

Classically, mechanistic interpretability [Sharkey et al., 2025, Mueller et al., 2024, Ferrando et al., 2024, Elhage et al., 2021, Olah et al., 2020] aims to reverse engineer an entire model [Huben et al., 2024, Elhage et al., 2022], or *circuits* implemented by the model to solve particular tasks [Wang et al., 2023a]. *Model diffing* offers an alternative method by focusing on *changes* induced by fine-tuning. Since fine-tuning typically involves far less compute than the pre-training phase that establishes general knowledge and generic circuitry, its resulting modifications are expected to be limited in scope. This targeted nature suggests model diffing could be a *more tractable* approach to mechanistic interpretability than the full model analysis, while still providing valuable insights into core features of a model’s behavior.

^{*}Equal contribution. Order randomized.

¹We open-source our [code](#), [training library](#), [models](#), [wandb runs](#) and a [demo notebook to explore latents](#).

Model diffing might indeed be incredibly useful. The process of fine-tuning a model is what makes it *useful* as a tool or agent. Better understanding the mechanisms that give reasoning models [DeepSeek-AI et al., 2025, OpenAI et al., 2024] heightened capabilities as compared to base or chat models might allow us to debug their failures and improve them. Fine-tuning also often introduces a number of problematic behaviors, for example, sycophancy [Sharma et al., 2023]. Future AI safety and alignment concerns [Greenblatt et al., 2024, Meinke et al., 2025, Betley et al., 2025] may emerge specifically in fine-tuned models. For example, long-horizon RL could incentivize models to exploit reward signals and act deceptively. Model diffing could allow us to detect this.

Prior model diffing research has investigated how models change during fine-tuning [Shah et al., 2023, Lindsey et al., 2024, Bricken et al., 2024, Prakash et al., 2024, Lee et al., 2024, Jain et al., 2024, Khayatan et al., 2025, Thasarathan et al., 2025, Wu et al., 2024, Mosbach, 2023, Merchant et al., 2020, Hao et al., 2020, Kovaleva et al., 2019, Du et al., 2025, Minder, 2024]. While these studies have hypothesized that fine-tuning primarily shifts and repurposes existing capabilities rather than developing new ones, conclusive evidence for this claim remains elusive. Model diffing remains a nascent field that lacks established consensus and mature analytical tools. Much prior work has leveraged ad-hoc techniques for understanding how models change in narrow ways (e.g. focusing on a particular circuit), or have been on toy model. It is unclear whether prior approaches would scale to understanding the kinds of fine-tuning large models actually undergo.

Recently, Lindsey et al. [2024] introduced the **crosscoder**, a novel and scalable tool for model diffing. Crosscoders build on the popular sparse autoencoder (SAE) [Huben et al., 2024, Bricken et al., 2023, Yun et al., 2021], which has shown promise for interpreting a model’s representations by decomposing activations into a sum of sparsely activating dictionary elements. There are many variants of crosscoders; the variant we are concerned with in this paper concatenates the activations of the base and chat-tuned model residual streams and trains a shared dictionary across this activation stack. Thus, for each dictionary element (aka "latent", corresponding to one concept), the crosscoder learns a pair of latent directions - one corresponding to the base model and one to the chat-tuned model. Crosscoders can thus potentially identify which latents are novel to the fine-tuned model, which are novel to the base-model, and which are shared. We term these sets chat-only, base-only, and shared respectively. Lindsey et al. [2024] identify chat-only latents by looking at the norm of the latent directions – if the latent direction of the base model has zero norm, this indicates that the latent is chat-only.

In this work, we critically examine the crosscoder and identify two theoretical limitations of its training objective, that may lead to falsely identified chat-only latents (Section 2.2):

1. **Complete Shrinkage:** The sparsity loss can force base latent directions to zero norm, even when they contribute to base model reconstruction.
2. **Latent Decoupling:** The crosscoder may represent a shared concept using a chat-only latent when it is actually encoded by a different combination of latents in the base model, as the crosscoder’s sparsity loss treats both representations as equivalent.

We develop an approach called *Latent Scaling* to detect spurious chat-only latents, inspired by Wright and Sharkey’s [2024] SAE scaling (Section 2.3), and demonstrate that the above issues occur in practice. While the norm-based metric from Lindsey et al. [2024] appears to identify a clean trimodal distribution of base-only, shared and chat-only latents, we show that this is an artifact of the loss function rather than a meaningful distinction. Our conclusion is that the crosscoder loss does not actually have an inductive bias that helps to learn better model-only latents. Nonetheless, we demonstrate that crosscoders trained with BatchTopK loss [Bussmann et al., 2024] exhibit robustness to the above issues (Section 3.1) and identify a larger number of genuine model-specific latents. We show that in the BatchTopK crosscoder, the norm-based metric successfully identifies causally relevant latents by measuring their ability to reduce the prediction gap between base and chat model. In contrast, this metric fails in the L1 crosscoder, where Latent Scaling becomes necessary to identify the truly causally relevant latents. Finally, we outline that the chat-only latents found by the BatchTopK crosscoder are highly interpretable (Section 3.3), revealing key aspects of chat model behavior such as the role of chat template tokens, persona-related questions, detection of false information, and various refusal related mechanisms.

Overall, we show that using BatchTopK loss overcomes the described limitations of L1-trained crosscoders, validating them as a useful tool for understanding fine-tuning effects in large language models.

2 Methods

Note: For reference, we provide a comprehensive glossary of key terms and mathematical notation introduced through the paper in Appendix A.

2.1 Crosscoder architectures

To build intuition, the crosscoder’s goal is to learn a dictionary of interpretable concepts (latents) that can explain the activations of both models. It consists of an encoder and a decoder. The encoder takes the activations of the base and chat models and projects them into a shared high-dimensional sparse space, where each dimension corresponds to a potential concept. The decoder then reconstructs each model’s activations using model-specific representations for each latent, combining them according to the sparse encoding. The key insight is that while both models share the same sparse encoding for a given input, the crosscoder learns separate decoder representations for each model, allowing concepts to have different importance or manifestation in each model.

More formally, let x be a string and $\mathbf{h}^{\text{base}}(x), \mathbf{h}^{\text{chat}}(x) \in \mathbb{R}^d$ denote the activations at a given layer. The encoder computes a sparse encoding $f_j(x) \in \mathbb{R}_{\geq 0}$ for each latent $j \in \mathcal{J} = \{1, \dots, D\}$. The decoder then reconstructs the activations as:

$$\tilde{\mathbf{h}}^{\text{base}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{base}} + \mathbf{b}^{\text{dec,base}} \quad \text{and} \quad \tilde{\mathbf{h}}^{\text{chat}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{chat}} + \mathbf{b}^{\text{dec,chat}} \quad (1)$$

where $\mathbf{d}_j^{\text{base}}, \mathbf{d}_j^{\text{chat}} \in \mathbb{R}^d$ are the model-specific decoder representations and $\mathbf{b}^{\text{dec,base}}, \mathbf{b}^{\text{dec,chat}} \in \mathbb{R}^d$ are decoder biases. The crosscoder minimizes reconstruction errors $\epsilon^{\text{base}}(x) = \mathbf{h}^{\text{base}}(x) - \tilde{\mathbf{h}}^{\text{base}}(x)$ and $\epsilon^{\text{chat}}(x) = \mathbf{h}^{\text{chat}}(x) - \tilde{\mathbf{h}}^{\text{chat}}(x)$ while enforcing sparsity.

We examine two sparsity mechanisms. The L1 crosscoder [Lindsey et al., 2024] adds an L1 penalty to the loss:

$$\mathcal{L}_{\text{L1}}(x) = f_j(x) (\|\mathbf{d}_j^{\text{base}}\|_2 + \|\mathbf{d}_j^{\text{chat}}\|_2) \quad (2)$$

The BatchTopK crosscoder [Bussmann et al., 2024] instead enforces L0 sparsity by selecting only the top nk latents with highest scaled activation $f_j(x_i)(\|\mathbf{d}_j^{\text{base}}\|_2 + \|\mathbf{d}_j^{\text{chat}}\|_2)$ across a batch of n strings.² More details on crosscoder implementation can be found in Section B.

2.2 Decoder norm based model diffing and its problems

To leverage crosscoders for model diffing, we can exploit the observation that while latent activations $f_j(x)$ are shared between models, the decoder vectors $\mathbf{d}_j^{\text{chat}}$ and $\mathbf{d}_j^{\text{base}}$ are unique to each model.

To leverage crosscoders for model diffing, we exploit that while the sparse encoding $f_j(x)$ is shared between models, the decoder representations $\mathbf{d}_j^{\text{chat}}$ and $\mathbf{d}_j^{\text{base}}$ are model-specific. When a latent is important for both models, both decoder representations need substantial norms for reconstruction. Conversely, a latent specific to the chat model will have $\|\mathbf{d}_j^{\text{chat}}\|_2 \gg 0$ while $\|\mathbf{d}_j^{\text{base}}\|_2 \rightarrow 0$, as the base decoder has no use for this latent.

We quantify this using the relative norm difference $\Delta_{\text{norm}} : \mathcal{J} \rightarrow [0, 1]$ from [Lindsey et al., 2024]:

$$\Delta_{\text{norm}}(j) = \frac{1}{2} \left(1 + \frac{\|\mathbf{d}_j^{\text{chat}}\|_2 - \|\mathbf{d}_j^{\text{base}}\|_2}{\max(\|\mathbf{d}_j^{\text{chat}}\|_2, \|\mathbf{d}_j^{\text{base}}\|_2)} \right) \quad (3)$$

Intuitively, $\Delta_{\text{norm}} = 1$ indicates a pure chat-only latent (base decoder has zero norm), $\Delta_{\text{norm}} = 0$ indicates a pure base-only latent, and $\Delta_{\text{norm}} \approx 0.5$ suggests equal importance in both models. As shown in Figure 1, we classify latents as *base-only* (0–0.1), *chat-only* (0.9–1.0), or *shared* (0.4–0.6).

Are *chat-only* latents really chat-specific? If a latent only contributes to one model, the norm of the decoder must tend to zero for the other model. But is the converse true? Specifically, we ask the

²During inference, a learned threshold θ zeroes out latents below it. See Equation (14).

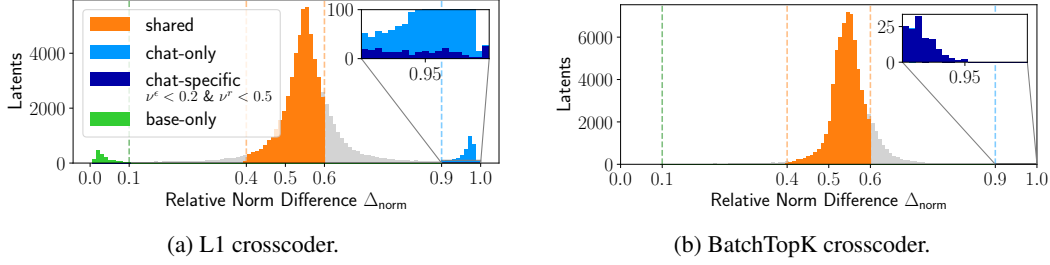


Figure 1: Histogram of decoder latent relative norm differences (Δ_{norm}) between base and chat Gemma 2 2B models [Riviere et al., 2024], for both the L1 crosscoder (left) and the BatchTopK crosscoder (right). A value of 1 means the decoder vector of a latent for the base model is zero, indicating the latent is not useful for the base model (*chat-only* latents). A value of 0 means the chat model’s decoder vector has a norm of zero (*base-only* latents). Values around 0.5 indicate similar decoder norms in both models, suggesting equal utility in both models (*shared* latents)³. We also show the *chat-only* latents that are truly chat-specific and that are not affected by Complete Shrinkage (error ratio $\nu^e < 0.2$) and Latent Decoupling (reconstruction ratio $\nu^r < 0.5$) – the *chat-specific* latents. Most of the L1 crosscoder *chat-only* latents suffer from these issues.

question: if a latent has decoder norm zero in the base model, is it necessarily chat-specific? We focus on the *chat-only* set, as it will contain features that emerged during chat-tuning.

Reasons to doubt *chat-only* latents. There are reasons to suspect *chat-only* latents might not be chat-specific. Firstly, both qualitative and quantitative analysis of L1 crosscoder latents reveals a relatively low percentage of interpretable latents within the *chat-only* set (See Section 3.3). More worryingly, inspection of the L1 crosscoder loss (Equation (2)) uncovers two theoretical issues that could result in latents j , which are defined by their decoder vectors $\mathbf{d}_j$ and activation function f_j , being classified as *chat-only*, despite their presence in the activations of the base model:

1. **Complete Shrinkage:** When the contribution of latent j is smaller in the base model than in the chat model, L1 regularization can force $\mathbf{d}_j^{\text{base}}$ to zero despite its presence in the base activation. Consequently, ϵ^{base} contains information attributable to latent j . This is similar to “shrinkage” or “feature suppression” in SAEs [Jermyn et al., 2024, Wright and Sharkey, 2024, Rajamanoharan et al., 2024].
2. **Latent Decoupling:** a *chat-only* latent j is also present in the base activations but is reconstructed by other base decoder latents. In this case, the base reconstruction $\mathbf{h}^{\text{base}}$ contains information that could be attributed to latent j . See Section D for an illustrative example.

Why BatchTopK crosscoders might fix this. The BatchTopK crosscoder may address both Complete Shrinkage and Latent Decoupling issues that affect the L1 crosscoder. The key difference lies in their respective loss functions and optimization objectives.

For the L1 crosscoder, the loss function in Equation (2) includes an L1 regularization term that directly penalizes the norm of decoder vectors. This creates pressure to shrink decoder norms toward zero when a latent’s contribution is minimal, potentially causing Complete Shrinkage even when the latent has some explanatory power. In contrast, the BatchTopK crosscoder uses a different sparsity mechanism. Rather than penalizing all decoder norms, it selects only the top k most active latents per sample during training. This approach has two important advantages:

1. **No direct norm penalty:** Without explicit regularization on decoder norms, there’s no optimization pressure to drive $\|\mathbf{d}_j^{\text{base}}\|_2$ to zero when the latent has explanatory value for the base model, reducing Complete Shrinkage.
2. **Competition between latents:** The top- k selection creates competition among latents, discouraging redundant representations. This helps prevent Latent Decoupling by making it inefficient to maintain duplicate latents that encode the same information.

³We observe larger activation norms in the chat model, which shifts our distribution rightward, revealing that the chat model amplifies the norm of representations shared with the base model.

The BatchTopK approach thus creates an inductive bias toward learning more genuinely chat-specific latents, as the model must efficiently allocate its limited "budget" of k active latents. This should result in fewer falsely identified *chat-only* latents and a cleaner separation between truly model-specific and shared features.

2.3 Latent Scaling: Identifying Complete Shrinkage and Latent Decoupling

To empirically investigate whether Complete Shrinkage and Latent Decoupling occur, we introduce *Latent Scaling*, which measures how well a supposedly *chat-only* latent can explain base model activations. We achieve this by finding the optimal scale for latent j to best reconstruct the base activations:

$$\beta_j^{\text{base}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n \|\beta f_j(x_i) \mathbf{d}_j^{\text{chat}} - \mathbf{h}^{\text{base}}(x_i)\|_2^2 \quad (4)$$

This least squares problem has an efficient closed-form solution⁴. For a chat-specific latent, we would expect $\beta_j^{\text{base}} \approx 0$ as the latent shouldn't help explain base activations at all. However, due to superposition [Elhage et al., 2022], even genuinely chat-specific latents might correlate with other features, resulting in $\beta_j^{\text{base}} > 0$. To account for this, we measure chat specificity using a ratio that compares how well the latent explains each model $\nu_j = \beta_j^{\text{base}} / \beta_j^{\text{chat}}$ where β_j^{chat} is computed analogously using $\mathbf{h}^{\text{chat}}(\cdot)$ instead of $\mathbf{h}^{\text{base}}(\cdot)$. A value near zero indicates a chat-specific latent, while a value near one suggests the latent is equally present in both models.

While this ratio efficiently identifies spurious *chat-only* latents, it doesn't tell us *why* they're spurious: it conflates Complete Shrinkage and Latent Decoupling. To distinguish between these failure modes, we leverage the fact that the crosscoder decomposes base activations $\mathbf{h}^{\text{base}}$ into its reconstruction ($\tilde{\mathbf{h}}^{\text{base}}$) and what it fails to reconstruct (ϵ^{base}):

1. If Complete Shrinkage occurred, the latent's information should appear in the reconstruction error ϵ^{base} , because the latent's base decoder is shrunk to zero instead of reconstructing the activation. This is captured by the error ratio $\nu_j^{\epsilon} = \beta_j^{\epsilon, \text{base}} / \beta_j^{\epsilon, \text{chat}}$.
2. If Latent Decoupling occurred, the latent's information should appear in the reconstruction $\tilde{\mathbf{h}}^{\text{base}}$, having been captured by other base model latents. This is measured by the reconstruction ratio $\nu_j^r = \beta_j^{r, \text{base}} / \beta_j^{r, \text{chat}}$.

These additional β values are computed using the same approach as Equation 4, but replacing $\mathbf{h}^{\text{base}}$ with either the error or reconstruction terms⁵.

3 Results

We replicate the model diffing experiments by Lindsey et al. [2024] using the open-source Gemma-2-2b (base) and Gemma-2-2b-it (chat) models [Riviere et al., 2024]. We train L1 and BatchTopK crosscoders on the middle layer (13) activations of both models⁶, collected on a mixture of both web and chat data. To ensure a fair comparison, we choose hyperparameters for both crosscoders to reach an L0 of 100. For details on the training process, see Section K.

In Figure 1, we present the histogram of Δ_{norm} between base and chat for both the L1 and BatchTopK crosscoders. At first glance, the L1 crosscoder identifies substantially more *chat-only* latents than the BatchTopK crosscoder. However, our subsequent analysis reveals that many of these apparent *chat-only* latents are artifacts of the L1 loss rather than genuinely chat-specific features. Refer to Section L for additional empirical details on the crosscoders.

3.1 Demonstrating Complete Shrinkage and Latent Decoupling

Analysing the L1 crosscoder. We compute the reconstruction and error ratios (ν_j^r and ν_j^{ϵ}), for all L1 crosscoder *chat-only* latents on 50M tokens from the training set. For calibration, we examine these

⁴The closed-form solution is derived in Section E.1 which also gives some intuition on the optimal β .

⁵See Section E.2 for exact implementation Section E.3 for verification of correlation between ν values and actual reconstruction improvement.

⁶We chose the middle layer as it's where we expect to find the richest representations [Skean et al., 2025].